

Feature: Free times

From build-plan: feature 5c

Branch: feature/05c-free-times

Status: verified. Whole feature seen and agreed by Frank 2026-10-02; steps 5c.1 to 5c.5 built, tested and reviewed step by step; every blocking review finding fixed, F-70 to F-73 last on 2026-10-02. The checkpoint for the final review.

Goal

For a service, and either one person the customer picked or "any available", work out the start times a customer can book: bookable hours minus bookings, time off and Google busy times, with the service's buffers, the business's notice and its horizon. Serve them on a public route the widget (feature 9) will call. Worked out on every request, never stored.

The third of feature 5's four parts. Nothing shows on screen yet. 5d books one of these times and checks it again first.

In scope

- The start-time step on a service: `booking_link.slotIntervalMinutes`, empty meaning "every service length" (decision 3, from the plan).
- The rules that turn one person's bookable hours, their busy time, standby and the rooms into start times, with buffers, notice and the horizon.
- The rule for who gets an "any available" booking (decision 2), as a function 5d calls when it books.
- One read that gathers everything for a service, a person or "any available", and a range of dates.
- The public route, and the people a customer can pick for a service.

Decisions

1. **Buffers may fall outside bookable hours** (Frank, 2026-10-01, B). Only the appointment itself must fit inside a bookable window. The buffers still block: the appointment plus both buffers must not touch anyone's busy time. Primo: window 7:30 to 8:30, a 60-minute estimate with 15 after, so 7:30 is offered and 8:30 to 8:45 only has to be free. 2. **"Any available" goes to whoever has the fewest bookings that day** (Frank, 2026-10-02, B). Among the people free at that time: the fewest booking commitments starting on that date in the business's time zone (the commitment's start, so its buffer before included; time off does not count); ties go by name, then id. A room goes to the first free one by name, then id. "Any available" offers a time when at least one person (and a room, when one is needed) is free then. 3. **Start times repeat every service length by default; the owner can change it** (the plan). Stored on

`booking_link` as `slotIntervalMinutes`, null meaning the service's length. Counted from the start of each bookable window: a 9:00 to 12:00 window with 75-minute facials offers 9:00, 10:15 and, if it fits, nothing at 11:30 (11:30 + 75 runs past 12:00).

Chosen in this spec as the simplest safe option (reversible, recorded):

4. **A calendar that cannot be read never reads as free.** With "any available", that person is left out of the answer and one warning line is logged. When the customer picked that person, the route answers 503 `unavailable` ("Times cannot be read right now. Try again shortly."), so a broken calendar is never shown as an empty week. 5. **One request covers at most 31 dates** (the business's calendar dates, both ends included), clamped to today through today plus the horizon. One Google read per person per request. 6. **Daylight saving:** a start time that does not exist on the clock that day (the hour skipped in spring) is not offered; a repeated hour (autumn) uses its first occurrence. Lengths are real minutes. 7. **The public route names the people a customer can pick:** id and name only, the ones `findServiceResources` offers, by name. Nothing else about a person, and never whether they are on standby or connected to Google.

Out of scope

- Booking a time, holding it, the lead, the booking row, the Google event (5d).
- Setting the start-time step, skills, rooms or standby from a screen (12).
- The widget and its layout (9). Showing times in the customer's own time zone (the widget's job; the route answers in instants plus the business's zone).
- Rate limiting public routes (not in the plan; noted for 9, the first public traffic).
- Part-day standby (5b decided standby is dated).

Build loop

Steps are built one at a time on `feature/05c-free-times`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"`, `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

Build steps

- ☒ **5c.1 The start-time step on a service.**
 - `packages/shared/db/booking-tables/booking-link-table.ts` gains `slotIntervalMinutes` (integer, nullable, no default) and a check `booking_link_slot_interval_check` (null or > 0). Migration `0012_booking_link_slot_interval.sql`, generated from `packages/shared`.

- The dev seed gives one clinic service an explicit step (Chemical Peel, 30 minutes, starts every 15) and leaves the rest empty, with find-or-make so a second run changes nothing.
- **Done when** `db:migrate` builds a fresh `scheduleads_dev` and `db:seed` runs twice; backend tests prove an empty step is allowed, 15 is stored, and 0 and -5 are refused by the database; the backend and frontend builds pass.
- ☒ **5c.2 The free-time rules.** No database, every rule tested.
 - `backend/lib/local-time/` (one helper per file, after review F-60): `localTimeToMoment`, a business date plus a minute of the day in its time zone, to an instant, null when that time does not exist (decision 6); and `localDate`, `addDays`, now shared with feature 2's bookable hours.
 - `backend/lib/scheduling/apply-free-times-rules.ts`: for one person, from their resolved bookable hours (`ResolvedBookableHoursType`), the service (length, buffers, step), their busy blocks (commitments and Google), their standby dates, the rooms (null = no room check, else each room's busy blocks and standby dates), a date range and `now`: the start instants they can take, sorted. A start is offered when the appointment fits inside one bookable window (decision 1); it is at least the notice after `now`; its date is inside the horizon, not closed, not a standby date; the appointment plus both buffers overlaps none of the person's busy blocks; and, when a room is needed, at least one room is free over the same span and not on standby. Busy blocks are half-open: a block ending at 10:00 does not block 10:00.
 - **Done when** saved tests prove: Primo's 7:30 with the 15 after outside the window, and 7:30 refused when 8:30 to 8:45 is busy; the step (empty, then 15 minutes); notice and horizon; closed and one-off dates; standby; a needed room, none free, and no room check; touching busy blocks; the spring and autumn clock changes in America/Denver (Alberta stopped changing its clocks in 2026, so Edmonton has none to test).
- ☒ **5c.3 Who gets "any available".** No database.
 - `backend/lib/scheduling/choose-any-available.ts`: given a start time, the people free then with their booking count that day, and the free rooms, returns the person and the room (decision 2), or null when nobody is free.
 - `backend/lib/scheduling/count-bookings-that-day.ts` (added while building): each person's booking count on a date, from the commitment rows 5c.4 reads.
 - **Done when** saved tests prove: fewest bookings wins; time off does not count as a booking; a tie goes by name, then id; the first free room by name; nobody free gives null; a needed room with none free gives null.
- ☒ **5c.4 Free times for a service.**
 - `backend/lib/scheduling/find-free-times.ts`: `findFreeTimes({ organizationId, bookingLinkId, personId, fromDate, toDate, now })`. `personId` null is "any available". Reads the service, `findServiceResources` (5b), `findStandbyDates` (5b), `resolveBookableHours` per person (2), `findCommitments` for the people and rooms (5a), and `getBusyTimes` per person (3), over the range widened by the buffers. Applies 5c.2 per person; "any available" is the union of the people's times. Answers `{ timezone, people, startTimes`

} , or null when the service is missing, inactive or another business's, or the picked person is not offered for it. A picked person's unreadable calendar throws `CalendarUnavailableError` ; with "any available" that person is left out and `warnConnectFailed` -style one-line warning logged (decision 4).

- **Done when** database tests on the seeded clinic prove: a picked practitioner's times; "any available" as the union; a held commitment (with `holdTime`) removes the times it covers, buffers included; a standby date removes that person on that date only; a needed room taken removes the time; a person from another business answers null; a person whose connection needs reconnecting throws when picked and is left out of "any available".
-  **5c.5 The public route.**
 - `GET /public/:slug/booking-links/:bookingLinkId/times?from=YYYY-MM-DD&to=YYYY-MM-DD&person=<id>` in `backend/routes/public-booking-links-routes.ts` , `person` omitted for "any available". The query is checked by a shared Zod schema (`free-times-query-validation-schema.ts` in `booking-links-validation-schemas/`). The business comes only from the slug, as the other public routes do.
 - `RefusalCodeType` gains `unavailable` .
 - **Done when** route tests on `clinic-dev` prove: 200 with the timezone, the pickable people and start times; "any available" when `person` is left out; 400 for a bad date, `to` before `from` , more than 31 dates, a malformed person id; the same 404 for an unknown business, an inactive service and a person not offered for the service; 503 `unavailable` when the picked person's calendar cannot be read; the response never carries `organizationId` , standby or calendar details; `npm run build --workspace=frontend` passes (the typed client sees the route).

Files / areas

- `packages/shared/db/booking-tables/booking-link-table.ts` , `packages/shared/migrations/0012_booking_link_slot_interval.sql` , `packages/shared/scripts/seed-dev.ts` .
- `packages/shared/zod-validation/booking-links-validation-schemas/` (the query schema, exported through `index.ts`).
- `backend/lib/local-time/` : the clock helpers, one per file (5c.2).
- `backend/lib/scheduling/` : `apply-free-times-rules.ts` , `choose-any-available.ts` , `count-bookings-that-day.ts` , `find-free-times.ts` , tests beside each.
- `backend/lib/calendar/` : `CalendarUnavailableError` if no existing error fits (5c.4 checks).
- `backend/lib/errors/refuse.ts` , `backend/routes/public-booking-links-routes.ts` and its test.

Data / contracts

booking_link.slotIntervalMinutes: integer, nullable, no default. Null = start times every `durationMinutes`. Check `booking_link_slot_interval_check: slotIntervalMinutes is null or slotIntervalMinutes > 0`. Not on the public booking-link responses.

applyFreeTimesRules (pure): inputs as in 5c.2, output `Date[]` sorted ascending, unique. A start instant `s` is offered for one person when:

- its local date `d` is in `[max(fromDate, today), min(toDate, today + horizonDays)]`, not in `closedDates`, not a standby date of the person;
- `d`'s windows are the one-off date's windows if `d` has one, else the weekday's; some window `[a, b)` has `a <= m` and `m + duration <= b`, and the real end `s + duration` is no later than the moment the clock shows `b` when that moment exists (review F-59, decision 6), where `m` is the start's minute of the day and `m = a + k * step` for a whole `k >= 0` (`step = slotIntervalMinutes ?? durationMinutes`);
- `s >= now + minimumNoticeMinutes`;
- the span `[s - bufferBefore, s + duration + bufferAfter)` overlaps none of the person's busy blocks;
- rooms null, or some room not on standby on `d` whose busy blocks the span does not overlap.

chooseAnyAvailable (pure): `(people: { resourceId, name, bookingsThatDay }[], rooms: { resourceId, name }[] | null) => { personId: string; placeId: string | null } | null`. Inputs are only the free ones; people and rooms sorted inside, never trusted to arrive sorted.

countBookingsThatDay (pure): `(commitments: { resourceId, kind, startsAt }[], date: YYYY-MM-DD, timezone) => Map<resourceId, count>`. Counts only `kind: "booking"` rows whose `startsAt` (buffer before included) falls on `date` in the business's zone. 5d reads the same rows, so the room rule it also needs is one helper to share (review F-64, for 5d's spec).

findFreeTimes answers:

```
{
  timezone: string; // the business's IANA zone
  people: { id: string; name: string }[]; // who the customer can pick, by name
  startTimes: string[]; // ISO 8601 instants in UTC, ascending, unique
}
```

The route: `GET /public/:slug/booking-links/:bookingLinkId/times`.

| Query | Rule | |---|---| | `from`, `to` | required, `YYYY-MM-DD` real dates, `from <= to`, at most 31 dates | | `person` | optional, a resource id (same rule as other ids); omitted = any available |

| Status | Body | |---|---| | 200 | the `findFreeTimes` answer | | 400 | `bad_request` | | 404 | `not_found`, one identical answer for every "not here" | | 503 | `unavailable`, only when the picked person's calendar cannot be read |

Testing

Backend Vitest, with the local `scheduleads_dev` migrated and seeded. 5c.2 and 5c.3 are pure and need no database. 5c.4 and 5c.5 use the seeded clinic, add their own rows and remove them, and refuse any database that is not local and `*_dev`. Google is never called in tests: a person with no connection has no busy times, and the unreadable case is a connection row marked `needs_reconnect`.

Notes for the AI

- Read time with `findCommitments` over the range widened by the largest buffer on each side, so a buffer reaching outside the range still sees what it touches.
- `resolveBookableHours` already applies the horizon, closed dates, holidays and one-off dates for the business or a person; do not re-implement them.
- Standby comes from `findStandbyDates`; rooms are read the same way (5b allowed a place on standby and reads it as hidden that day).
- `getBusyTimes` returns `[]` with no connection and throws when the connection needs reconnecting or the read fails; it never answers "free".
- From 5a: three or more holds on one person at the same instant can exhaust `holdTime`'s retries. 5c only reads; 5d owns holding.
- Public routes give one identical 404 for every "not here" and take the business only from the slug.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/booking-tables/booking-link-table.ts` gained `slotIntervalMinutes` (integer, nullable) and `booking_link_slot_interval_check` (null or above zero). Empty means start times repeat every service length. Migration `0012_booking_link_slot_interval.sql` is generated and untouched.
- `scripts/seed-dev.ts` gives the clinic's Chemical Peel a 15-minute step and leaves every other service empty. Its find-or-make never updates an existing service, so a dev database migrated rather than rebuilt keeps the peel empty (F-58, open); the free-time tests build clinics of their own instead.
- `zod-validation/booking-links-validation-schemas/free-times-query-validation-schema.ts`: the question a widget asks. Real `YYYY-MM-DD` dates, the last not before the first, at most 31 dates counting both ends, and an optional person id of the same shape as a booking link id. Every refusal carries our own words, including a person asked for twice (F-71).

backend/lib/local-time

One helper per file (F-60), shared with feature 2's bookable hours: `clockAsUtc` (what the business's clock shows at a moment, as a number; the base of the rest, with one cached `Intl.DateTimeFormat` per zone), `localDate`, `addDays` (plain calendar arithmetic in UTC, so no clock change can shift it) and `localTimeToMoment` (a business date and minute of the day to an instant; null for the hour skipped in spring, the first of the two in autumn). Clock change tests use America/Denver: Alberta stopped changing its clocks in 2026 (tzdata 2026c), so Edmonton has no change left to test, and a server must carry that tzdata or newer.

backend/lib/scheduling

- `apply-free-times-rules.ts`, pure: one person's start instants for a range. The range is cut to today through today plus the horizon; each date uses its one-off windows or the weekday's; starts step from each window's start by the service's step or its length; only the appointment must fit the window (decision 1), and on the spring change its real end must not pass the moment the window ends (F-59). Then notice, closed and standby dates, the person's busy blocks over the appointment plus both buffers (half-open, so touching blocks do not clash), and a free room when the service needs one.
- `choose-any-available.ts`, pure, for 5d: among the people free at a start, the fewest bookings that day, ties by name then id; the first free room by name then id; null when nobody (or no needed room) is free. It sorts its own inputs rather than trusting their order.
- `count-bookings-that-day.ts`, pure, added while building 5c.3: bookings per person on one business date, counted on the date the commitment starts (buffer before included, F-63); time off never counts.
- `find-free-times.ts`: the one read for a service. It loads the active service inside the business, who is offered for it (5b) and the business's hours, answers null for anything not there, then cuts the dates to today through the business's horizon before reading anything (F-70; a person's rules never carry their own horizon). Taken time is read a day wider than the range on each side plus the buffers, so a buffer or a zone ahead of UTC still sees what it touches. People are read side by side, so several Google calendars cost about one call's wait. A picked person's unreadable calendar throws `CalendarUnavailableError` with the cause kept and one warning line; with "any available" that person is left out with a warning (decision 4).

backend/lib/calendar and errors

`calendar-unavailable-error.ts` is the one new error; `refuse.ts` gained the refusal code `unavailable`.

backend/routes

`public-booking-links-routes.ts` gained `GET /public/:slug/booking-links/:bookingLinkId/times`. The query goes through Hono's own `validator("query")` with the

shared schema, as the admin's client setup does, so the typed client in the frontend learns the question's shape with no new package. The business comes only from the slug, through the same `findBookableOrganizationId` as the other public routes. Answers: 200 with `{ timezone, people, startTimes }`, 400 `bad_request` with the reason, the one identical 404 for every "not here", and 503 `unavailable` only for a picked person's unreadable calendar; any other failure still reaches the general error path. `coding-standards.md` now allows a public answer to name the people a customer can pick, id and name only (F-72, decision 7).

Tests

Pure rules and helpers have their own tests beside them. `find-free-times.test.ts` builds small clinics of its own, so none depends on the dev seed's state, and fakes Google with a stubbed `fetch`. The route tests run against the seeded `clinic-dev` (Deep Cleansing Facial: Ana, Mei, Sofia; Luis is the person not offered), use dates a week ahead of today on the clinic's clock, stub `fetch` so no dev login's real calendar can be reached (F-73), and test the broken calendar on a throwaway clinic so the seed is never changed. Every step's saved tests were shown able to fail by breaking the code on purpose, one piece at a time.

Carried forward

- F-74 (open): with "any available", if every offered person's calendar is unreadable, the route answers 200 with no times, which reads as fully booked. The spec does not say what should happen when nobody is left; decide in 5d or feature 9.
- F-62 (open): the rules run per person on the event loop, about 75 ms for the clinic's daytime hours; worth a cap or a rate limit when the widget opens to the public (feature 9 or 12).
- F-64 (open): one shared "which rooms are free" helper for 5d.
- F-58 (open): the seed does not update an existing peel's step.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-52, F-58, F-62, F-64 and F-74 stay in the live ledger.

5c/F-59 [P3] closed - On the spring clock change an appointment can run past the end of its bookable window

File: backend/lib/scheduling/apply-free-times-rules.ts:68 **Found:** 2026-10-02 by independent review of step 5c.2 (scope: cb29f51..b5bf8c4; lenses: all) **Why it matters:** The window fit is checked in clock minutes (`minute + durationMinutes <= endMinute`), while decision 6 says lengths are real minutes. On the day the clock skips an hour, a start before the skip whose appointment crosses it ends one hour later on the clock than the check assumes. Probe: America/Denver, Sunday 2027-03-14, window 1:30 to

3:00, a 60-minute service: 1:30 (08:30Z) is offered, and it ends at 09:30Z, which is 3:30 on the clock, 30 minutes after the window closes. The code matches the contract formula as written, so this is a gap between the contract and decision 6, not a broken contract. Only windows that span the skipped hour (around 2am, one day a year) are affected, so the practical risk is very low. **Suggested fix:** Either also require the real end to be no later than the window's end moment (`localTimeToMoment(date, window.endMinute)`), falling back to the clock check when that is null), or record in the spec that the fit is in clock minutes and accept it. **Resolution:** Fixed in 5c.2 review fixes: a start must also end, in real minutes, by the window end as a moment (when that moment exists); test "the spring change cannot stretch an appointment past its window" (Denver, 2027-03-14, 1:30 to 3:00, 60 minutes, now offers nothing), shown able to fail. Closed by independent review of step 5c.3 (2026-10-02): the check at `apply-free-times-rules.ts:79` compares the real end with the window end as a moment; a scratch copy with that check disabled fails exactly this test (it offers 2027-03-14T08:30Z). Autumn and midnight window ends resolve to real moments (first occurrence, minute 1440), so no new refusal or offer appears; a window ending inside the skipped hour falls back to the clock count, as its comment says. No new defect.

5c/F-60 [P3] closed - local-time.ts holds three exports and is now imported across areas, against one file per export

File: `backend/lib/scheduling/local-time.ts:36` **Found:** 2026-10-02 by independent review of step 5c.2 (scope: `cb29f51..b5bf8c4`; lenses: all) **Why it matters:** The backend standard is kind, then area, then one file per export, and a helper used across several areas goes in a shared place. `local-time.ts` exports `localDate` , `addDays` and `localTimeToMoment` ; it is the only backend lib file besides `auth-server.ts` with more than one exported function. Moving `localDate` and `addDays` here also makes `lib/bookable-hours/apply-bookable-hours-rules.ts:6` (feature 2) import from `lib/scheduling` (feature 5), so the older area now depends on the newer one. Behaviour is unchanged (the bookable-hours tests pass), so this is about finding things by file name, which is how Frank navigates. **Suggested fix:** Split into one file per function (for example `local-date.ts` , `add-days.ts` , `local-time-to-moment.ts`), and put the two that both areas use where shared helpers go, or say in the spec that `local-time.ts` is a deliberate exception. **Resolution:** Fixed in 5c.2 review fixes: split into `backend/lib/local-time/` with one export per file (`clock-as-utc`, `local-date`, `add-days`, `local-time-to-moment`) and a test beside each; feature 2 and 5c both import from there, so neither reaches into the other area. Closed by independent review of step 5c.3 (2026-10-02): `backend/lib/scheduling/local-time.ts` is gone, `backend/lib/local-time/` holds one export per file, nothing imports the old path (grep), and the full backend suite (256 tests) passes. One small inaccuracy above: `clock-as-utc.ts` has no test of its own; it is covered through the three helpers' tests. No new defect.

5c/F-61 [P3] closed - A few edges of the free-time rules have no test: the room over the buffers, a block ending at the start, the last horizon date, a window ending at midnight

File: `backend/lib/scheduling/apply-free-times-rules.test.ts:134` **Found:** 2026-10-02 by independent review of step 5c.2 (scope: `cb29f51..b5bf8c4`; lenses: all) **Why it matters:** All the Done-when cases are covered and pass, but these mutations would still pass the suite: a room checked over the appointment

only instead of the span with both buffers; a busy block that ends exactly at a start being treated as blocking (only the "starts as the appointment ends" side is tested); `date < lastDate` in place of `<=` , because the horizon test proves Oct 12 is out but never that the last day (Oct 9) is in; and a window ending at minute 1440. Probes show the current code handles all four correctly, so this is a guard for later edits, not a defect. **Suggested fix:** Add four small cases: a room whose busy block touches only the after-buffer; busy ending at 9:00 with a 9:00 start offered; a horizon whose last date has hours and is offered; a 22:00 to 24:00 window offering 22:00 and 23:00. **Resolution:** Fixed in 5c.2 review fixes: five cases added (a room checked over the buffers, busy time ending at the start, the horizon's last date, a window to midnight, and F-59's spring case); each shown able to fail by breaking its rule on purpose. Closed by independent review of step 5c.3 (2026-10-02): re-broken in a scratch copy, each case failed as claimed (room checked over the appointment only; busy end treated as blocking with `>=` ; `date < lastDate` ; the spring check removed). No new defect.

5c/F-63 [P3] closed - A booking is counted on the day its buffer starts, not the day its appointment starts, and the counting helper is not in the spec

File: backend/lib/scheduling/count-bookings-that-day.ts:14 **Found:** 2026-10-02 by independent review of step 5c.3 (scope: 4d6d1ce..4858600; lenses: all) **Why it matters:** The comment says "A booking belongs to the day it starts on", but a booking commitment's `startsAt` already includes the buffer before (commitment-table.ts:24, hold-time.ts:16). Probe: an appointment at 00:10 on Tuesday Oct 6 in Edmonton with 15 minutes before has a commitment starting Monday 23:55, and `countBookingsThatDay` counts it on Monday, not Tuesday. The code matches decision 2's literal wording ("booking commitments on that date"), so this is a wording gap, and only bookings within a buffer of midnight are affected, so the practical risk is very low. Separately, `count-bookings-that-day.ts` is not named in the spec's step 5c.3, Files / areas, or Data / contracts, although 5d will need it to produce `bookingsThatDay` . **Suggested fix:** Correct the comment to say the day the commitment starts (buffer included), or say in decision 2 which start counts; and add `countBookingsThatDay(commitments, date, timezone)` to the spec's files and contracts so 5d calls it as specified. **Resolution:** Fixed in 5c.3 review fixes: the comment now says a booking belongs to the day its commitment starts on, buffer before included (only a buffer reaching back over midnight moves it), matching decision 2's wording; the spec's decision 2 says which start counts, and `countBookingsThatDay` is in step 5c.3, Files / areas and Data / contracts. Closed by independent review of step 5c.4 (2026-10-02): `count-bookings-that-day.ts:14-15` now says the commitment's start, buffer before included, which is what the code does (`localDate` of `startsAt`) and what decision 2 now says; the helper and its contract are in the spec's step 5c.3, Files / areas and Data / contracts. No new defect.

5c/F-65 [P3] closed - Step and decision history in the new code comments

File: backend/lib/scheduling/count-bookings-that-day.ts:2 **Found:** 2026-10-02 by independent review of step 5c.3 (scope: 4d6d1ce..4858600; lenses: all) **Why it matters:** The comments standard says no history in code comments (step numbers, finding numbers): that lives in the build log. `count-bookings-that-day.ts:2` says "(5c.3)" and `choose-any-available.ts:1` says "(decision 2, Frank, 2026-10-02)".

Feature references elsewhere in the backend are common, but a step number and a dated decision are the history the standard names. **Suggested fix:** Drop "(5c.3)" and the date; keep the rule itself in the comment ("the fewest bookings that day, ties by name then id"). **Resolution:** Fixed in 5c.3 review fixes: the step number and the date are gone from both headers; the rule stays. Closed by independent review of step 5c.4 (2026-10-02): count-bookings-that-day.ts and choose-any-available.ts carry no step number or date, and the rule is kept. The same kind of comment remains in test files (choose-any-available.test.ts:6 and :15, and the new find-free-times.test.ts:1), recorded as F-69.

5c/F-66 [P3] closed - No test subtracts Google busy time or reaches the edges of the read window

File: backend/lib/scheduling/find-free-times.test.ts:1 (code: find-free-times.ts:85-89 and :129) **Found:** 2026-10-02 by independent review of step 5c.4 (scope: 577bab2..d6ef7d9; lenses: all) **Why it matters:** Google busy time is half of what this step subtracts, yet no test gives a person a Google busy block: every test person is unconnected or needs reconnecting. Mutation probes in a scratch copy: dropping `...googleBusy` from the busy list (line 129) passes all 8 tests, and so does narrowing the read window by a day on each side (`- DAY_MS` removed, `2 * DAY_MS` to `DAY_MS`). The second would miss a booking or Google busy time on an Edmonton evening (after 18:00 local is the next UTC day) and offer a time that is taken. The code is right today; nothing would catch a later edit. The other probes (union, picked throw, picked not offered, rooms, standby, busy per person, the warning line) each fail a test, and dropping the service's active check is a double guard `findServiceResources` also holds. **Suggested fix:** Add two cases: a connected person with Google faked the way `get-busy-times.test.ts` does (a stubbed fetch and a saved connection), whose busy block removes a start; and a commitment late on the last date (an evening window, after midnight UTC) that removes the time it covers. **Resolution:** Fixed in 5c.4 review fixes: two tests added, a person's faked Google busy time removing the time it covers, and taken time late on the last date (Tuesday in UTC) still seen; each shown able to fail by dropping Google's busy time and by narrowing the read window by a day. Closed by independent review of step 5c.5 (2026-10-02): both tests are in `find-free-times.test.ts` at ac66226 and pass.

5c/F-67 [P3] closed - A picked person's unreadable calendar leaves no log line, and the reason is dropped

File: backend/lib/scheduling/find-free-times.ts:118 **Found:** 2026-10-02 by independent review of step 5c.4 (scope: 577bab2..d6ef7d9; lenses: all) **Why it matters:** With "any available" the failure is logged with its reason (needs reconnecting, Google's status, a bad token key, a database fault). When the customer picked that person, the same failure becomes a bare `CalendarUnavailableError` with no `cause` and nothing logged, so 5c.5 answers 503 and whoever looks later cannot tell why; the route cannot log it either, since the reason is gone. For a one-person business every request is a picked one. **Suggested fix:** Log the same one warning line before throwing, or pass the original as `cause` (the constructor taking `ErrorOptions`) so 5c.5 logs it with `safeErrorReason` . **Resolution:** Fixed in 5c.4 review fixes: a picked person's unreadable calendar now logs the same one-line warning before throwing, and the original error travels as the `CalendarUnavailableError`'s cause; the test checks both. Closed by independent review of

step 5c.5 (2026-10-02): the warning and the cause are still in find-free-times.ts at ac66226, and the test passes.

5c/F-68 [P3] closed - A range whose last date is before its first throws a database error, and 5c.5's clamp can make one

File: backend/lib/scheduling/find-free-times.ts:85 **Found:** 2026-10-02 by independent review of step 5c.4 (scope: 577bab2..d6ef7d9; lenses: all) **Why it matters:** The read window is built from the raw dates, so when `fromDate` is more than about two days after `toDate` the window is inverted and `findCommitments` fails ("Reading time failed: database error 22000", probed in a scratch copy with Oct 12 to Oct 5), a 500; were that read to pass, `getBusyTimes` would refuse the range too, which a picked person turns into a misleading 503. The route's Zod rule (`from <= to`) stops a raw inverted request, but decision 5 then clamps the range to today through today plus the horizon: a request wholly in the past (Sep 1 to Sep 15, clamped to start today) or wholly past the horizon inverts after clamping. Unclamped, Google and the commitments are also read for dates the rules will discard. **Suggested fix:** In 5c.5, answer an empty clamped range with no start times before calling `findFreeTimes`, and add that case to its Done when; or have `findFreeTimes` answer `startTimes: []` when `fromDate > toDate`, and say which in the spec's contract. **Resolution:** Fixed in 5c.4 review fixes: `findFreeTimes` answers no times, without reading anything, when `fromDate` is after `toDate`; test added and shown able to fail. Closed by independent review of step 5c.5 (2026-10-02): the guard still answers before any read at ac66226; the route never sends an inverted range (Zod refuses one), and its unclamped reads are recorded as F-70.

5c/F-69 [P3] closed - Step numbers in test file comments

File: backend/lib/scheduling/find-free-times.test.ts:1 **Found:** 2026-10-02 by independent review of step 5c.4 (scope: 577bab2..d6ef7d9; lenses: all) **Why it matters:** The comments standard says no history in code comments, step numbers included. The new test's header says "every reader 5c.4 gathers", and two from step 5c.3 remain in `choose-any-available.test.ts`: 6 ("step 5c.3's simulation") and :15 ("the commitment rows 5c.4 will read"). **Suggested fix:** Drop the step numbers and keep what each comment says (for example "every reader free times gathers", "the commitment rows the free-time read gathers"). **Resolution:** Fixed in 5c.4 review fixes: the step numbers are gone from the two test files' comments. Closed by independent review of step 5c.5 (2026-10-02): no step number in any comment under `backend/lib`, `backend/routes` or the new shared schema at ac66226.

5c/F-70 [P2] closed - The route passes the asked dates unclamped, so a valid date near 9999 is a 500 and far-off dates still read Google

File: backend/routes/public-booking-links-routes.ts:123 (reads: backend/lib/scheduling/find-free-times.ts:87-91) **Found:** 2026-10-02 by independent review of step 5c.5 (scope: 5368e0c..ac66226; lenses: all) **Why it matters:** Decision 5 says a request is "clamped to today through today plus the horizon"; the route instead sends the dates as asked and relies on the rules to clamp the answer. The reads before the rules are built from the raw dates. Probed through the real app against `clinic-dev` : ?

`from=9999-12-30&to=9999-12-30` (and `9999-12-31`) answers 500, with "Reading time failed: database error 22009" from `findCommitments`, because the read window ends two days later, in year 10000, which Postgres refuses; `9999-12-29` answers 200. `z.iso.date` accepts every year 0000 to 9999, so a stranger reaches this with a query the schema calls valid, against the contract's 200/400/404/503. The same unclamped window means `?from=2099-01-01&to=2099-01-31` (200, no times) still reads every candidate's commitments and, for a connected person, Google's busy times, for dates the rules then discard (F-68 already named this; its fix covered only the inverted range). **Suggested fix:** Clamp before reading: in `findFreeTimes`, from the business hours already resolved there, take `max(fromDate, today)` and `min(toDate, today + horizonDays)` in the business's zone (or the largest horizon among the candidates, if a person's can be longer), then let the existing `fromDate > toDate` guard answer no times without reading. Add route tests for a range wholly past the horizon (200, no times) and for `9999-12-31` (not 500). Correct the route comment at line 123 to match. **Resolution:** Fixed 2026-10-02: `findFreeTimes` now cuts the dates to today through the business's horizon (a person's rules never carry their own) before any read; a range left empty answers no times unread. Route test for `9999-12-30` to `9999-12-31` (200, no times) added; removing the clamp makes it fail with the 500. Closed by the final independent review of feature 5c (2026-10-02): at `bb2526d` `find-free-times.ts:81-86` cuts the dates to today through the business horizon (at most 365 days, `availability_rule_horizon_check`) before the commitments, standby and Google reads that follow, so the read window can no longer reach year 10000; the route test for `9999-12-30` to `9999-12-31` passes and the route comment at :123 matches. No new defect.

5c/F-71 [P3] closed - A repeated person in the query answers Zod's own English message

File: `packages/shared/zod-validation/booking-links-validation-schemas/free-times-query-validation-schema.ts:17` **Found:** 2026-10-02 by independent review of step 5c.5 (scope: 5368e0c..ac66226; lenses: all) **Why it matters:** Hono's query validator turns a repeated key into an array. Probed through the real app: `?from=2026-10-09&to=2026-10-12&person=a&person=b` answers 400 with `"Invalid input: expected string, received array"`, the library's wording, where every other refusal on the public routes is the project's own sentence. A repeated `from` or `to` is fine ("Use a real date, YYYY-MM-DD.") because `z.iso.date` carries its message for every issue; the `.regex` message on `person` covers only the pattern, not the type. The status is right; only the message leaks the library. **Suggested fix:** Give the string itself the message, `z.string({ error: "That is not a person id." }).regex(...)`, and add a schema test with `person: ["a", "b"]`. **Resolution:** Fixed 2026-10-02: the person rule carries its message on the string itself, so a person asked for twice reads "That is not a person id.". Schema test added; removing the message makes it fail. Closed by the final independent review of feature 5c (2026-10-02): `free-times-query-validation-schema.ts:18` gives the string itself the message, and the schema test with `person: ["a", "b"]` passes. No new defect.

5c/F-72 [P3] closed - The coding standard says a public answer carries nothing about people, but the times route names them

File: `blueprint/context/coding-standards.md:213` **Found:** 2026-10-02 by independent review of step 5c.5 (scope: 5368e0c..ac66226; lenses: all) **Why it matters:** The public-route rule ends "The answer never

carries `organizationId` or anything about people or logins". Decision 7 deliberately lets the times route name the people a customer can pick (id and name), and the route does. The code follows the spec, so the standard is now wrong, and a later reviewer reading it would flag the route, or a later route would be built to the stricter line. The rule in AGENTS.md is to correct a wrong plan file before the next step builds on it. **Suggested fix:** Amend the line to: never `organizationId` or anything about logins; about people, only the id and name of those offered for a service, never standby, hours or calendar details (decision 7). **Resolution:** Fixed 2026-10-02: coding-standards.md now says a public answer names, of people, only the id and name of those a customer can pick for a service, never standby, a calendar or contact details (decision 7). Closed by the final independent review of feature 5c (2026-10-02): coding-standards.md:212-214 now allows exactly decision 7's id and name, and the route's answer (find-free-times.ts:70-77 reads id and name only) and its test (no organizationId, standby, calendar, google or connection in the text) match it. No new defect.

5c/F-73 [P3] closed - The route tests call the seeded clinic with no guard against a real Google connection

File: backend/routes/public-booking-links-routes.test.ts:249 **Found:** 2026-10-02 by independent review of step 5c.5 (scope: 5368e0c..ac66226; lenses: tests) **Why it matters:** The spec says Google is never called in tests. The free-times route tests read the seeded `clinic-dev`, whose facial practitioners have no connection today, so nothing reaches Google; but nothing enforces it. The dev database already carries a real connected Google calendar on `painting-dev` (status `connected`, made from the dashboard), and the seed links `owner@example.com` to Sofia, one of the three facial practitioners. Once that login connects a calendar the same way, these tests send Sofia's real tokens to Google on every run, and their answers depend on her real calendar and on Google being reachable (an expired grant turns the picked-person test for her into a 503). `find-free-times.test.ts` avoids this by stubbing `fetch` to throw on any URL. **Suggested fix:** In the "free times for a service" describe, stub `fetch` to throw (as find-free-times.test.ts does), or assert in its `beforeAll` that no `clinic-dev` person has a calendar connection and fail with "the seed clinic has a calendar connected". **Resolution:** Fixed 2026-10-02: the free-times route tests stub fetch to throw for their whole block, so a calendar connected by a dev login can never reach Google; it would fail the test loudly instead. Closed by the final independent review of feature 5c (2026-10-02): public-booking-links-routes.test.ts:288 stubs fetch to throw in the block's beforeAll and unstubs in its afterAll; backend has no vitest config, so `unstubGlobals` is off and the stub holds for every test in the block. No new defect.

Independent review

Status: passed **Target commit:** bb2526df457b661cf73f585cc2133f18f0b11e22 **Base commit:** bf53ee6f9a79fd9fcafc09731283acc78e458f48 **Base ref:** main **Spec hash:** 9ebfbdbdf41a9ed4df0f8ef5a3f26b1e9cc04ad3b2d9f56e10aa83fd8ef8b499 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-02T19:01:27Z **Workflow:** regular **Check**

required: no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-02T19:05:13Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain --untracked-files=all` : pass (HEAD, merge base and spec hash match the request; only review.md differed)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass (typechecks the typed client against the new times route)
- `npm run test --workspace=@scheduleads-app/shared` : pass (10 files, 75 tests)
- `npm run test --workspace=backend` : pass (27 files, 282 tests, against local `scheduleads_dev`)
- `npm run format:check` : pass
- `npm run db:generate --workspace=@scheduleads-app/shared` : pass ("No schema changes, nothing to migrate"; no files written, working tree unchanged afterwards)

Evidence

- Whole delta read: 37 files in bf53ee6..bb2526d (5c.1 to 5c.5 code, tests, migration 0012, seed, shared query schema, spec, standards, AGENTS.md and four skill wording changes).
- Tenant scope: the times route takes the business only from the slug (`findBookableOrganizationId`); every read in `find-free-times.ts`, `find-service-resources.ts`, `find-commitments.ts`, `find-standby-dates.ts`, `resolve-bookable-hours.ts` and `get-busy-times.ts` filters on that `organizationId`; a picked person must be in `offered.peopleIds`; route tests prove the identical 404 for another business's service and person.
- Decisions 1, 3, 6: `apply-free-times-rules.ts` fits only the appointment in the window, steps from each window start by `slotIntervalMinutes ?? durationMinutes`, skips non-existent clock times and caps the real end at the window end's moment; DB checks keep duration and step above 0, so the loop always advances.
- Decision 2 and contracts: union of per-person times in `find-free-times.ts`; `chooseAnyAvailable` and `countBookingsThatDay` match the Data / contracts signatures and are tested (5d wires them).
- Decision 4: picked person's unreadable calendar logs and throws `CalendarUnavailableError` (cause kept), route maps only that to 503; "any available" logs and leaves the person out (see F-74).
- Decision 5: schema caps 31 dates; `findFreeTimes` clamps to today through the business horizon (at most 365 days) before any read, so one Google read per person per request over at most 34 days.
- Decision 7 and the standard: response carries timezone, people (id, name), `startTimes` only; test checks no `organizationId`, standby or calendar words.

- Tests: no skipped, focused or todo tests in the delta; Google is stubbed to throw in both database test files.

Findings

- F-74 [P3] open: "any available" answers 200 with no times when every candidate's calendar is unreadable, the empty week decision 4 rules out for a pick.
- F-70, F-71, F-72, F-73: re-examined and closed.
- Still open from earlier steps and carried: F-62 [P3] (per-request clock conversions), F-64 [P3] (room rule for 5d), F-58 [P3].

Remaining risk

- The public times route has no rate limit (out of scope, noted for feature 9): each stranger request makes one Google read and one `lastCheckedAt` write per connected candidate, and repeats `resolveBookableHours` (about three queries) per person.
- F-62's synchronous rule cost grows with long windows and a small step on that same unthrottled route.
- `/check` not run (not required); no running-server or browser evidence, and no real Google calendar was read (stubbed in tests by design).